Chapter 3
Bitcoin Core: The Reference Implementation

Course Notes

CSE0469 Blockchain: Special Topics in SW Engineering

Yusuf Altunel

Based on Mastering Bitcoin 2e; Andreas M. Antonopoulos; O'Reilly, 2017

Content

- Bitcoin Development Environment
- Running a Bitcoin Core Node
- Bitcoin Core Application Programming Interface (API)
- Alternative Clients, Libraries, and Toolkits
- Windows Installation



Bitcoin: an open source project

Bitcoin Project

free to download

free to use

Usage

Open source

Free for any purpose

License

the source code licensed

Under open(MIT) license



Bitcoin: an open source project



Community

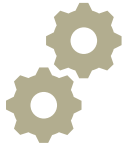
bitcoin is developed by an **open community**



Full-time Developers

a **dozen** developers

working on the code almost full-time



Part-time Developers

several dozen more on a part-time basis.



Bitcoin: an open source project

Statistics

By **2016:** more than **400** contributors

To the source code of Bitcoin core

Contribution to activities

Improve documentation

Translations

Participating to mailing lists

Contribution to development

Improving User Interface

Coding

Testing

Who can Contribute?

Anyone can contribute to the code

Check https://bitcoincore.org/en/contribute/



Satoshi Client

Famous paper

«Bitcoin: A Peer-to-Peer Electronic Cash System»

“Satoshi client”

1st version of Bitcoin implementation

known as “Bitcoin” or “Satoshi client”

Implemented

before the white paper

Improvements

Satoshi client has been heavily modified and improved



Bitcoin Core

«Client» to «Core»

Satoshi client evolved into **Bitcoin Core**

Bitcoin core

the authoritative reference

Aim

show how each part of the technology

to be implemented



Bitcoin Core Features

wallets

storing Bitcoin

transacting in Bitcoin

engine

transaction validation

block validation

Network node

a full network node

the peer-to-peer bitcoin network



Wallet in Bitcoin Core

Bitcoin Core Wallet

Implementation is included

just as a reference implementation

Not for use

not intended to be used

for real users or applications!

Actual Wallet

Advised to use BIP 39/32

A modern standard for walled implementations



Wallet in Bitcoin Core

BIP

stands for

Bitcoin Improvement Proposal.

BIP-32

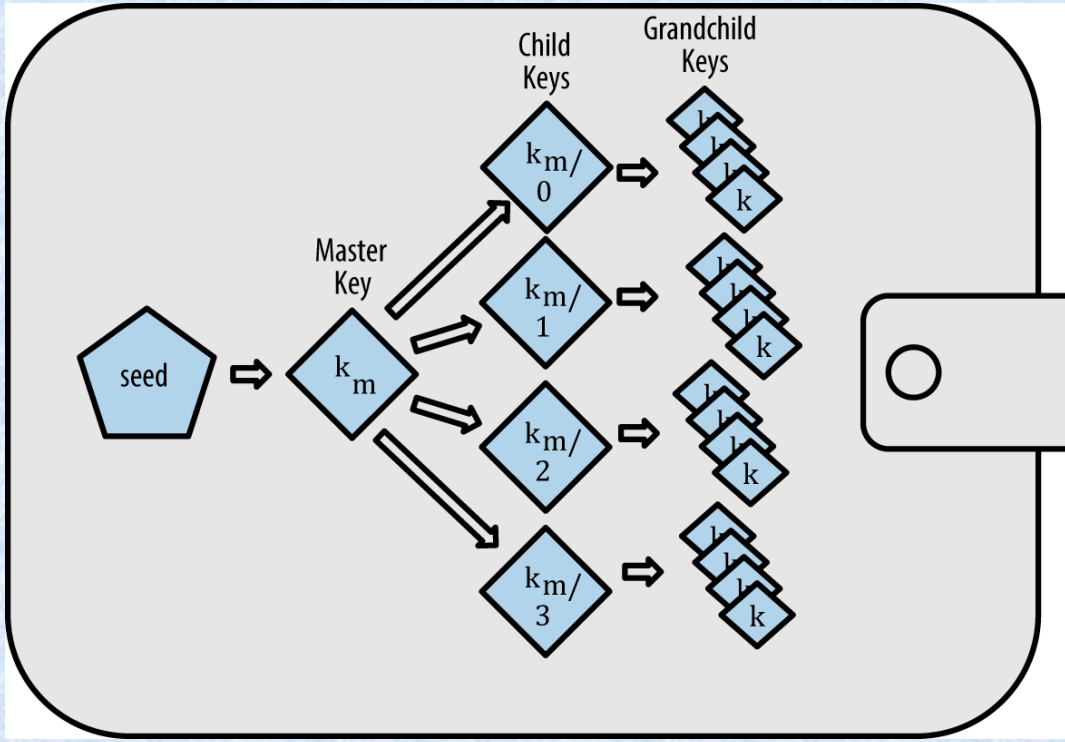
HD

hierarchical deterministic wallet

Features

multiple addresses for a Bitcoin

derived from a single seed



Wallet in Bitcoin Core

BIP-39

to manage recovery seed

and recovery words

Mnemonic

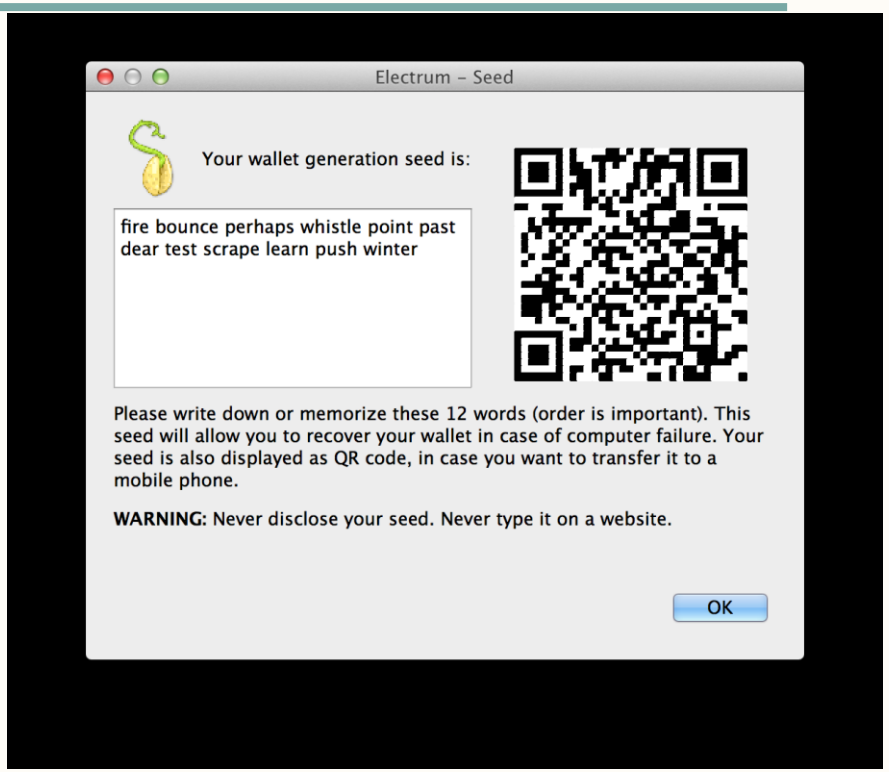
a mnemonic code

or mnemonic sentence

Usage

Mnemonic is used

to generate and recover keys



Bitcoin Developmen Environment



Developers

Set up dev environment



Environment

all the tools

libraries

support software

Download Bitcoin Core's Source Code

Download

- as a ZIP archive On the GitHub Bitcoin Page
- select Download ZIP
  https://github.com/bitcoin/bitcoin

Local copy

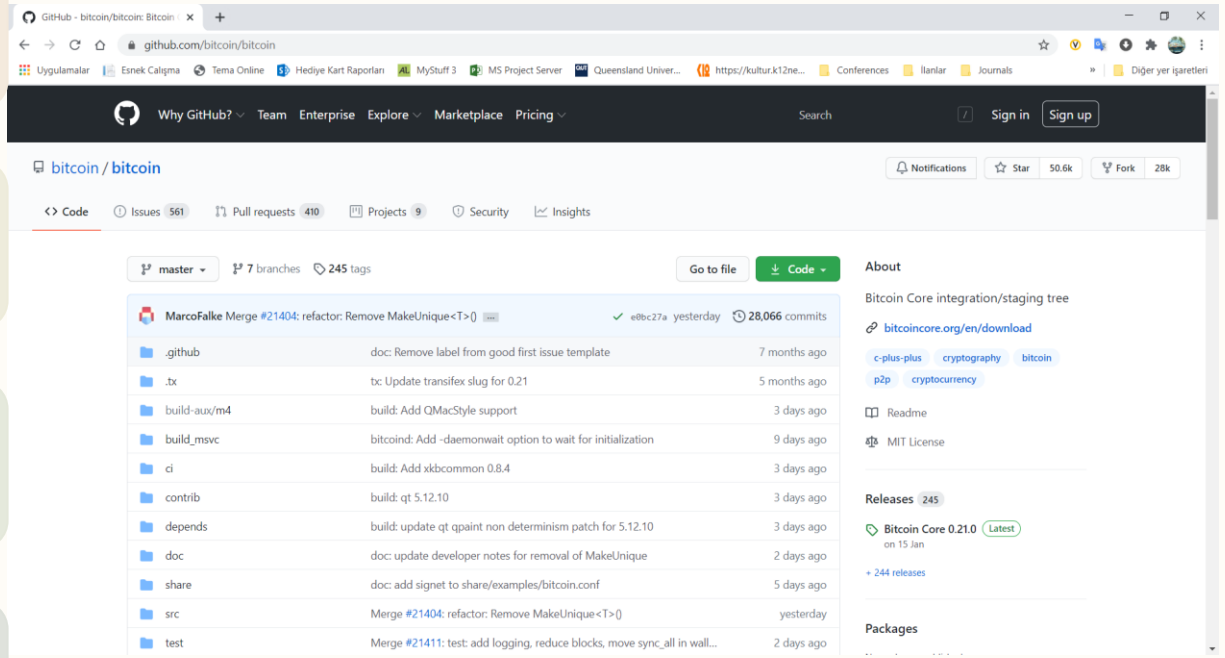
- Create local copy of source code
- Use git Command line
- $ git clone https://github.com/bitcoin/bitcoin.git

Git

- Distributed version control system

Notice!

- You need to install Git if you don't have it



Selecting a Bitcoin Core Release



Synchronization

By default, the local copy will be synchronized

with the most recent code



Synchronize before compile

Before compiling the code,

select a specific version

by checking out a release tag.



This will synchronize

the local copy

with a specific snapshot

of the code repository

identified by a keyword tag.



Tags

used by the developers

to mark specific releases of the code

by version number.

Running a Bitcoin Core Node



Node

Bitcoin’s peer-to-peer network composed of network “nodes,”



authoritative view

a direct and authoritative view of the bitcoin blockchain



Transactions

a local copy of all the transactions



Independent validation

independently validated by their own system.

Running a Bitcoin Core Node



voluteers

run mostly by volunteers



Business

build bitcoin applications



Independence

No need to rely on third party

to validate a transaction.



conribution

contribute to the bitcoin network

by making it more robust.

Resources



Full stack node

index all transactions

starting from 2009



Full copy

keep a full copy of the blockchain



Connection

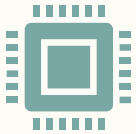
a permanently connected system



Resources

to process transactions

Minimum Requirements



Configuration

2 GB RAM

one-time download of about **400GB** of data

plus a further **5-10GB** per month

100 MB/s read/write speed



System

Desktop

Laptop

ARM chipsets>1GHz



Operating Systems

Desktop or laptop hardware

running recent versions of OS



Operating Systems

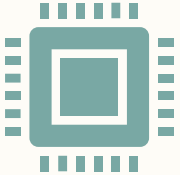
MS Windows

Mac OS X

Linux

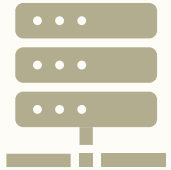
https://bitcoin.org/en/full-node#minimum-requirements

Various Solutions



Raspberry Pi

a $35 USD computer the size of a pack of cards)



VPS-*Virtual Private Server*

run bitcoin nodes on

rented servers

usually some variant of Linux.

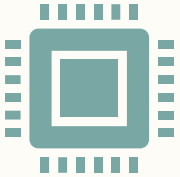


Cost

can be rented

for $25 to $50 USD per month.

Reasons to Run a Node



Development

on a bitcoin node

for programmable (API)



Access to Bitcoin

access to the network and blockchain.

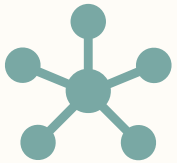


Transaction Validation

building applications

to validate transactions

Reasons to Run a Node



Support Bitcoin

Running a node

makes the network more robust



Serve

more wallets

more users

more transactions



Security

to not rely on any third party

to process or validate own transactions

Running Bitcoin Core for the First Time



Run bitcoind

typing **bitcoind** into the terminal



Configuration file

tells what is needed

to build a configuration file



Notice

it is recommended

you to set up the alerting mechanism.

Configuring the Bitcoin Core Node



Configuration

Edit the configuration file

in preferred editor

set the parameters,



Pasword

replace the password

with a strong password

as recommended by bitcoind



Notice

Do not use the password

shown in the book.



bitcoin.conf

Create a file inside the .bitcoin directory

under your user's home directory

named .bitcoin/bitcoin.conf



Account

provide a username and password:

rpcuser=bitcoinrpc

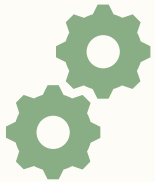
rpcpassword=CHANGE_THIS

Other Configurations



options

Bitcoin Core offers
more than 100 configuration options



What can be modified?

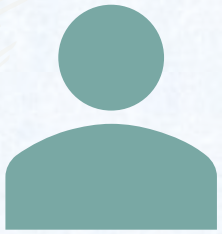
behavior of the network node
the storage of the blockchain
many other aspects of its operation



To see a listing of these options,

run bitcoind --help:

most important options



alertnotify

to send emergency alerts
to the owner of this node
usually by email



conf

An alternative location
for the configuration file

most important options



datadir

Select the directory and filesystem

to put all the blockchain data.



Default

this is the *.bitcoin* subdirectory

of your home directory.



Space

Make sure this filesystem

has several gigabytes of free space.

most important options



prune

Reduce the disk space requirements



Old Blocks

by deleting old blocks



Usage

if the full blockchain

cannot fit to the disk space

most important options



txindex

Maintain an index of all transactions



means

a complete copy of the blockchain

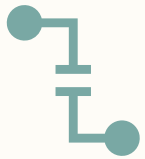


allows

to programmatically retrieve

any transaction by ID

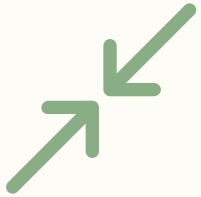
most important options



maxconnections

maximum number of nodes

to accept connections



Bandwith

reduce bandwidth consumption.



Useful

if there is a data cap

or pay by the gigabyte

most important options



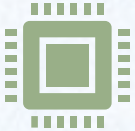
maxmempool

Limit the transaction memory pool to so many megabytes.



Use it

to reduce memory use of the node.



maxreceivebuffer/maxsendbuffer

Limit per-connection memory buffer to so many multiples of 1000 bytes.



Use

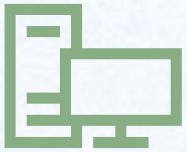
on memory-constrained nodes

most important options



minrelaytxfee

Set the minimum fee transaction you will relay



Below this value

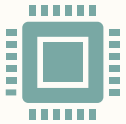
the transaction is treated as zero fee.



Use

to reduce the size of the in-memory transaction pool.

Bitcoin Core's Programmatic Interface



Interface

Bitcoin Core's API is a JSON-RPC interface.



RPC

Remote Procedure Call

calling remote procedures or functions



Connection

the Bitcoin Core node via a network protocol



HTTP

the network protocol is HTTP



HTTPS

for encrypted connections

Bitcoin Core's Programmatic Interface



curl

command-line HTTP client

to construct one of these JSON-RPC calls



The command

curl submits an HTTP request

to the local host

127.0.0.1



Default port

connecting to the default bitcoin port

8332



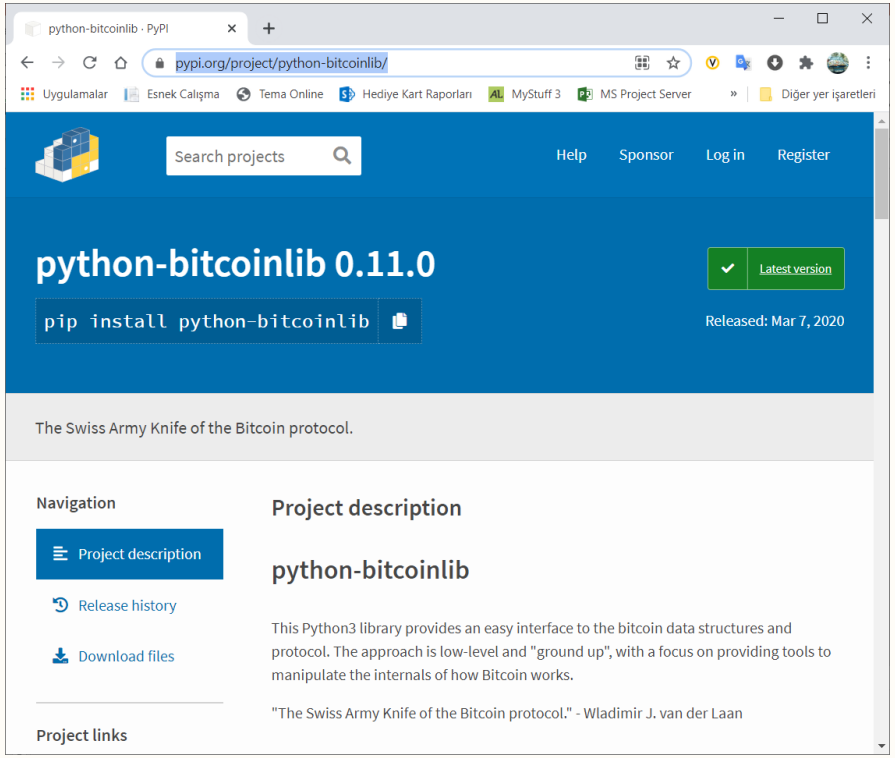
submitting a jsonrpc request

for the getinfo method

using text/plain encoding

```
$ curl --user myusername --data-binary '{"jsonrpc": "1.0", "id":"curltest",
"method": "getinfo", "params": [] }' -H 'content-type: text/plain;' http://
127.0.0.1:8332/
```

Pythonbitcoinlib library



Libraries

provides an easy interface

to the bitcoin data structures and protocol

Example Code

examples/ directory

chain to use

import bitcoin

bitcoin.SelectParams(NAME)

NAME

- 'testnet'
- 'mainnet'
- 'regtest'

https://pypi.org/project/python-bitcoinlib/

Download: https://pypi.org/project/python-bitcoinlib/#files

Alternative Clients, Libraries, and Toolkits



Bitcoin Core

The reference implementation of bitcoin



libbitcoin

Cross-platform C++ development toolkit, node, and consensus library



bitcoin explorer

Libbitcoin's command-line tool



picocoin

A C language lightweight client library for bitcoin by Jeff Garzik

– C/C++

JavaScript



bcoin

- A modular and scalable full-node implementation with API

Bitcore

- Full node, API, and library by Bitpay

BitcoinJS

- A pure JavaScript Bitcoin library for node.js and browsers

Java



bitcoinj

- A Java full-node client library

web3j

- Lightwieght Java & Android Library
- For integration with Etherium clients
- Notice: Not available in the book!

fabric-sdk-java

- a backbone for developing blockchain solutions
- under development from the Linux Foundation members
- Notice: Not available in the book!

Bits of Proof (BOP)

- A Java enterprise-class implementation of bitcoin
- Notice: Not a valid library anymore!

Python



python-bitcoinlib

- A Python bitcoin library, consensus library, and node by Peter Todd

pycoin

- A Python bitcoin library by Richard Kiss

pybitcointools

- A Python bitcoin library by Vitalik Buterin
- Notice: This library is not maintained anymore!

Ruby, Go, and Rust



Ruby

- *bitcoin-client*
- A Ruby library wrapper for the JSON-RPC API

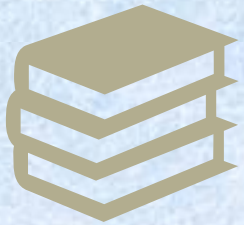
Go

- *btcd*
- A Go language full-node bitcoin client

Rust

- *rust-bitcoin*
- Rust bitcoin library for serialization, parsing, and API calls

C# & Objective-C



C#

[NBitcoin]

Comprehensive bitcoin library for the .NET framework



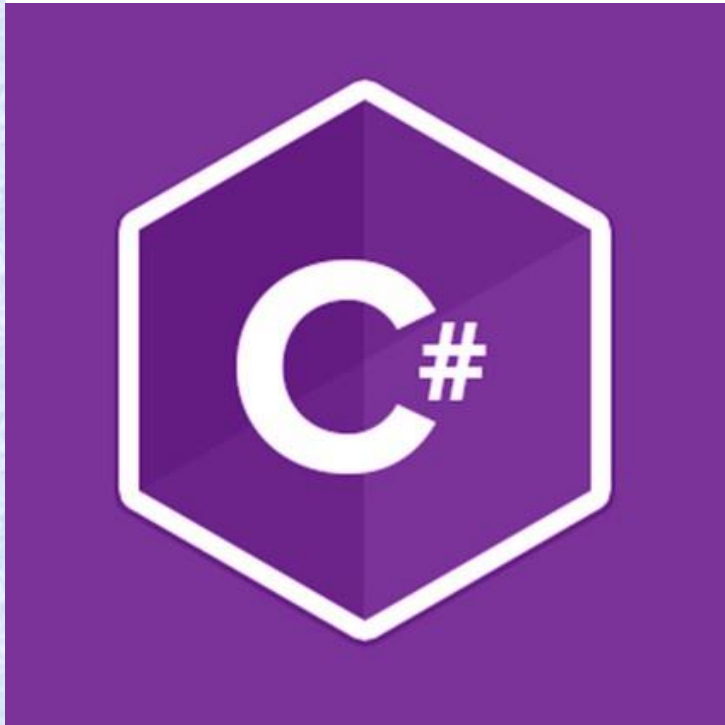
Objective-C

[CoreBitcoin]

Bitcoin toolkit for ObjC and Swift



Objective-C



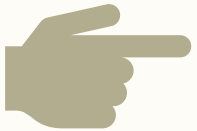
Instructions for Windows



Download

Go to the Bitcoin Core Download Page

Click the *Download Bitcoin Core*



Run installer

run downloaded installer file

Windows will ask you to confirm that you want to run it.

Click Yes and the Bitcoin installer will start.



Typical Windows installer

It’s a typical Windows installer,

it will guide you through the decisions you need to make about where to install Bitcoin Core.

Options

Core GUI

If you want to use the Bitcoin Core Graphical User Interface (GUI)

proceed to the Bitcoin Core GUI section below.

Bitcoin Core daemon

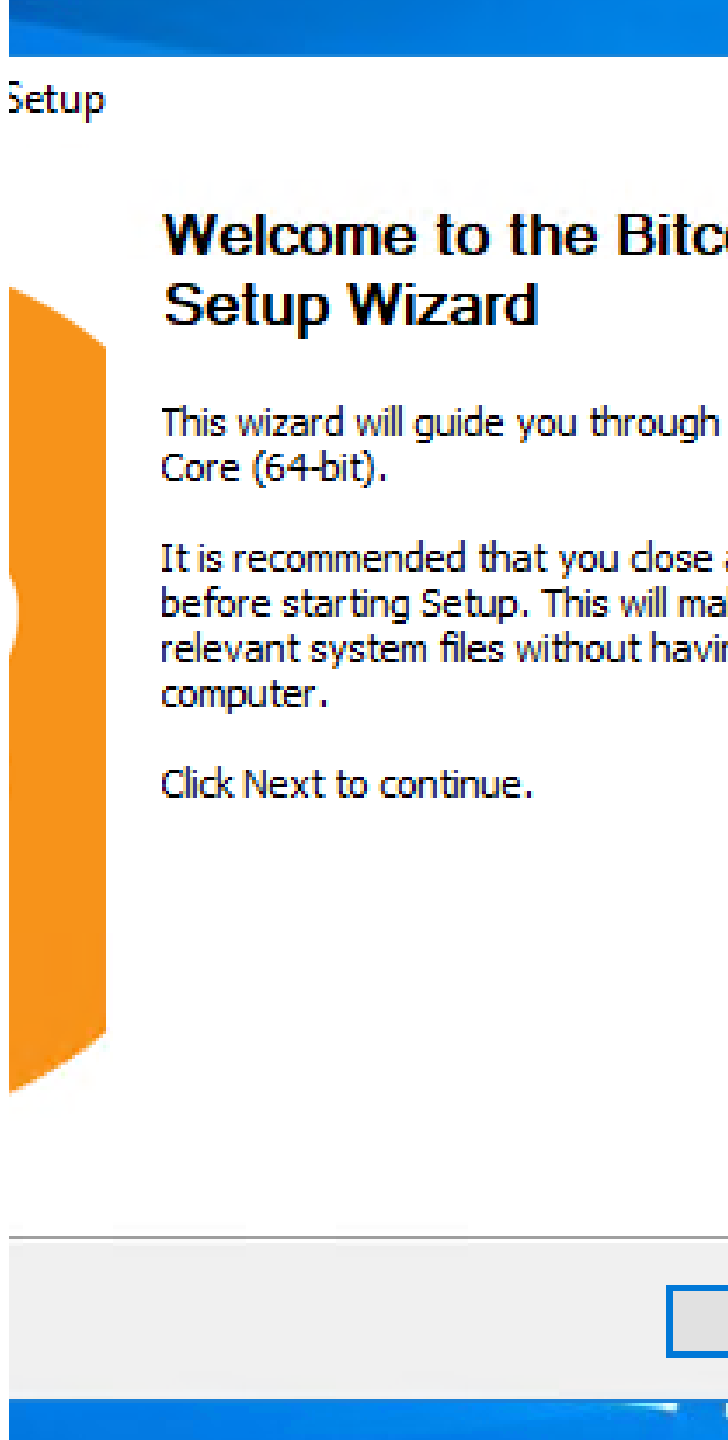
useful for programmers and advanced users

bitcoind: use the Bitcoin Core daemon

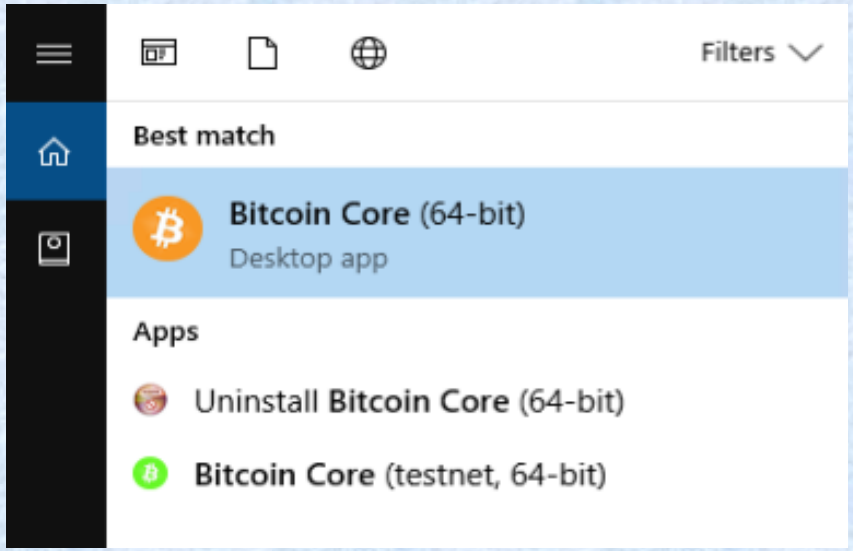
Both

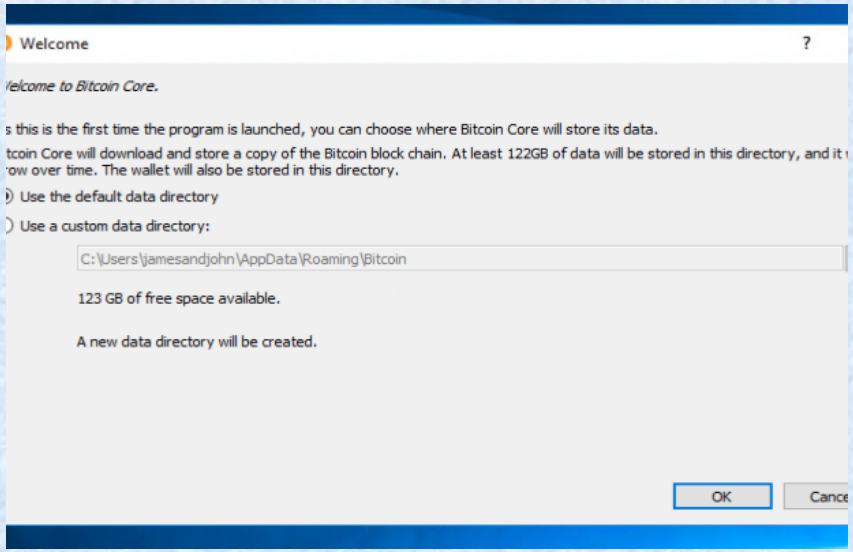
you **can't** run both the GUI and the daemon at the same time

using the same configuration directory



Bitcoin Core GUI





Install

- Press Windows key (⊞ Win)
- start typing “bitcoin”.
- When the Bitcoin Core icon appears click on it.

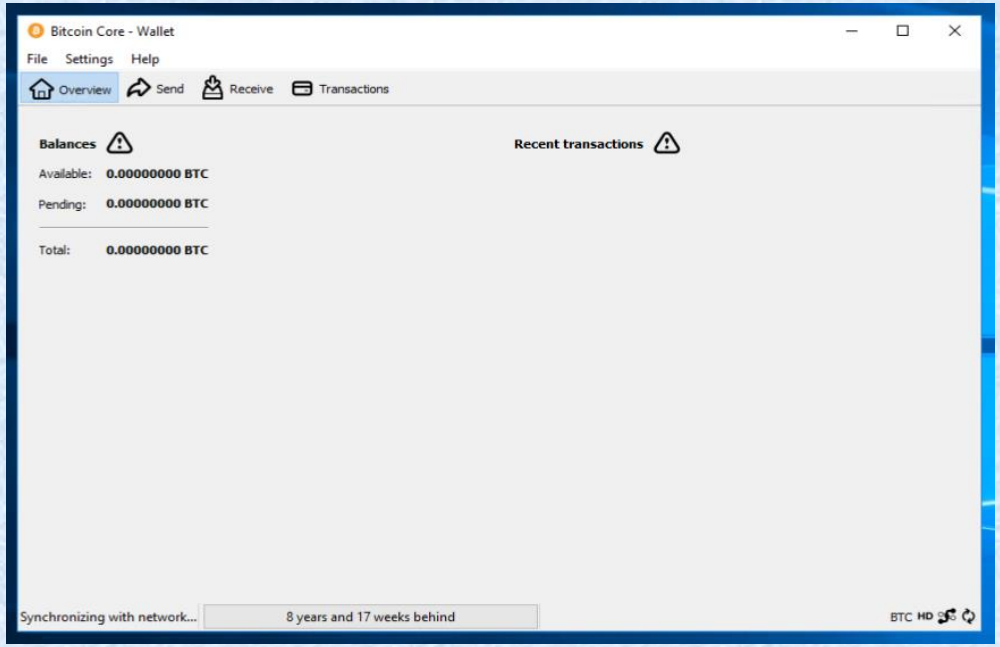
Chose a directory

- to store the Bitcoin block chain and your wallet.
- Unless you have a separate partition or drive you want to use,
- click Ok to use the default.

Firewall setting

- Your firewall may block Bitcoin Core from making outbound connections.
- It’s safe to allow Bitcoin Core to use all networks.

download the block chain



Block Chain Download

Bitcoin Core GUI will begin to download the block chain

takes at least **several days**

Consume CPU, Memery and Internet connection

Start/Stop

You can stop Bitcoin Core

at any time by closing it

it will resume

from the point where it stopped

the next time you start it.

Final Steps



Start to use

use Bitcoin Core as wallet

or just let it run

to help support the Bitcoin network



Automatic start

Optional:

Start Your Node At Login



Starting automatically

at login

makes it easy to contribute to the network.

Final Steps

1

Setting up Automatic Start

- tell Bitcoin Core GUI to start at login.

2

Setup

- While running Bitcoin Core GUI
- open the Settings menu and choose Options.

3

Main tab

- On the Main tab,
- click *Start Bitcoin on system login*.

4

OK

- Click the Ok button
- to save the new settings

Discussions

Asses what you got so far!

Noticible issues

What do you think on this chapter?

Do you think the bitcoin inspiring?



Interesting Details

What is the most interesting point?

Select and explain a critical issu in this chapter



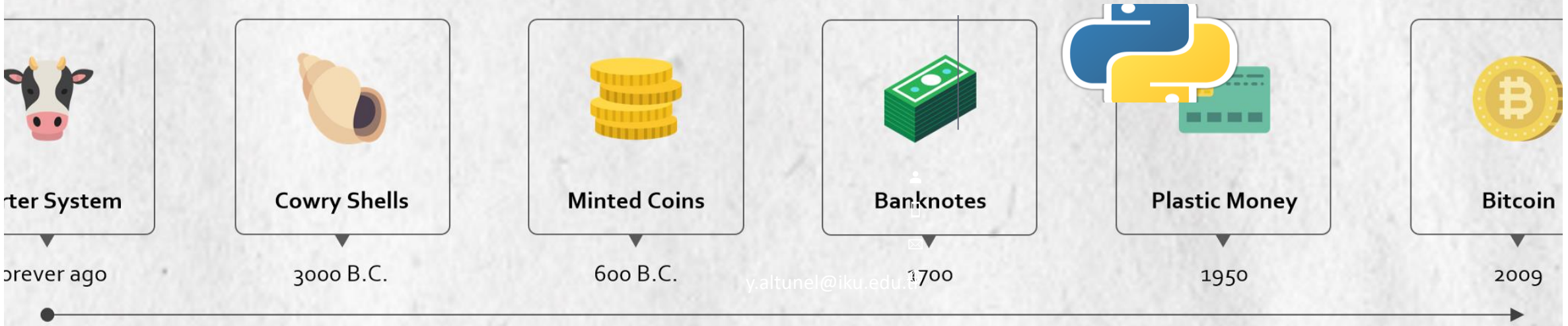
Unclarified issues

What are the points need further explanations?



Thank You

The History of Money



Yusuf Altunel, PhD

https://www.linkedin.com/in/yusuf-altunel-2070a31b/

https://cats.iku.edu.tr/portal/site/cd88fc16-3584-46f0-a539-890caf466d53